

University of Stuttgart
Germany
Institute of Industrial Automation and
Software Engineering

Risk-Annotated Spatial Scene Graph for Dy- namic Risk As- sessment in Autonomous Systems

Master Thesis TBD

**Submitted at the University of Stuttgart by
Fabin Fizek Sha**

Electrical Engineering

Examiner: Prof. Dr.-Ing. Andrey Morozov

Supervisor: Joachim Grimstad, M.Sc.

**External
Supervisor:** ()

30 October 2026



Abstract

Conventional risk assessment mechanisms are limited when the control strategy or operating environment of an autonomous system changes after deployment. This limitation is particularly relevant for systems operating in dynamic environments or modified through adaptive software components and over-the-air updates.

This thesis contributes to the development of a dynamic risk assessment mechanism by extending an existing scene graph generation framework. The original FastAPI-based implementation is ported to ROS 2 to improve portability and integration with robotic systems. To support operation in larger environments, the scene graph generation process is extended through the integration of SLAM-based spatial data and loop closure detection. In addition, a probabilistic object representation is developed to account for uncertainty in object perception and to provide more robust information for risk assessment.

The thesis further refines the output of the vision-language model to produce bounded and consistent scene graph information. The proposed extensions are implemented with consideration for near-real-time operation, enabling the generated risk-annotated scene graph to provide structured environmental context during system operation.

The framework is evaluated using multiple datasets, including a manually generated dataset collected with a Unitree Go1 robot in a university environment. Overall, this work improves the scalability, portability, and robustness of spatial risk-annotated scene graph generation and supports the development of dynamic risk assessment mechanisms for autonomous systems.

Keywords: dynamic risk assessment, risk-annotated scene graph, ROS 2, SLAM, loop closure detection, probabilistic object representation, vision-language model

Contents

Abstract	ii
List of Figures	v
List of Tables	vi
1. Introduction	1
1.1. Motivation	1
1.2. Relevance	1
1.3. Goal of the Thesis	2
2. Theoretical Background	3
2.1. Autonomous Systems and Safety	3
2.1.1. Human-Robot Collaboration	4
2.1.2. Conventional Risk Assessment and Risk Reduction	5
2.1.3. Existing Safety Measures in HRC	5
2.1.4. Impact of Artificial Intelligence	7
2.2. Robot Operating System	8
2.2.1. ROS as Robotic Middleware	8
2.2.2. ROS Communication Mechanisms	8
2.2.3. ROS 1 and ROS 2	9
2.3. Spatial Scene Graphs	10
2.3.1. Scene Graphs	10
2.4. Hydra-Based Spatial Scene Graph Generation	11
2.4.1. Input Data and Processing Overview	11
2.4.2. Layered 3D Scene Graph Representation	13
2.4.3. Metric-Semantic Mesh and Object Layer Construction	13
2.4.4. Places and Topological Free-Space Representation	14
2.4.5. Room Detection Algorithm	14
2.4.6. Loop Closure Detection and Scene Graph Optimization	15
2.4.7. Real-Time Architecture and Graph Visualization	16
2.4.8. Adaptation for RGB-D and IMU Input from a Real Robot	17
2.4.9. Relevance to the Proposed Framework	18

Contents

2.5. Risk-Annotated Scene Graph Framework Baseline	18
2.5.1. Information Flow in the Baseline Framework	19
2.5.2. FastAPI	20
2.5.3. SAM-Based Object Segmentation	21
2.5.4. Retrieval-Augmented Perception	21
2.5.5. Vision-Language Model Reasoning	22
2.5.6. Scene Graph Construction and Risk Annotation	23
2.5.7. Limitations of the Baseline Framework	23
2.6. Section4	25
2.7. Section5	25
2.8. Section6	25
2.9. Section7	25
3. Methodology	26
3.1. Baseline System and Proposed Extensions	26
4. Experiments	27
5. Conclusions	28
Bibliography	29
A. Example appendix chapter	32

List of Figures

2.1. Main challenges in HRC[10].	4
2.2. Illustrative representation of safety zones in a collaborative robot workspace.	6
2.3. Scene graph example[17].	10
2.4. Example of a layered 3D scene graph generated by Hydra, showing abstraction levels from metric-semantic 3D mesh to objects and agents, places, rooms, and buildings. Reproduced from [18].	12
2.5. Information flow of the existing risk-annotated scene graph framework. Adapted from [19].	19

List of Tables

1. Introduction

1.1. Motivation

Robotic systems are increasingly deployed across industrial, service, medical, and consumer domains. According to the International Federation of Robotics, annual installations of industrial robots increased from approximately 221,000 units in 2014 to 542,000 units in 2024, indicating that global robot demand in factories has roughly doubled over the last decade[1]. In the same period, the global operational stock of industrial robots increased from approximately 1.47 million units to 4.66 million units[1]. This development shows that robots are becoming a more widespread component of modern production and service environments.

At the same time, robotic systems are increasingly influenced by advances in artificial intelligence. The Stanford AI Index reports that 78% of organizations used AI in 2024, compared with 55% in the previous year [2]. In addition, global private investment in generative AI reached USD 33.9 billion in 2024. The International Federation of Robotics also identifies physical, analytic, and generative AI as key technological trends in robotics. These developments indicate that future robotic systems will increasingly combine physical operation with adaptive, data-driven, and software-defined capabilities[1].

This combination introduces new challenges for safety and risk assessment. Conventional risk assessment methods are typically performed before deployment and rely on assumptions about the system design, intended use, operating environment, and known hazards [3]. Although such methods remain essential, they are limited when autonomous systems operate in dynamic environments, receive over-the-air software updates, or use adaptive AI-based components. This creates a need for dynamic risk assessment mechanisms that can incorporate the current operational context through structured representations of relevant objects, spatial relationships, and risk-related information while operating at near real time speed. A spatial risk-annotated scene graph is therefore motivated as a means of providing environmental context and situational awareness to support risk assessment during system operation.

1.2. Relevance

Dynamic risk assessment requires more than the detection of individual objects. In autonomous systems, risk depends not only on whether an object is present, but also on its position, spatial

1. Introduction

relationships, uncertainty, and relevance to the current operational context. A scene graph provides a suitable representation for this purpose because it organizes perceived objects, spatial relations, and semantic information in a structured and machine-readable form.

With the increasing number and variety of autonomous systems, portability and scalability become important requirements for risk assessment mechanisms. Such mechanisms should not be limited to a specific robot, sensor setup, or software interface, but should be reusable across different autonomous platforms. At the same time, the scene graph must be continuously updated as the environment changes and should be scalable to larger spatial operating areas by incorporating information from multiple observations over time.

Uncertainty handling and runtime performance are also essential for practical deployment. Since object detection, localization, and classification can contain errors, a probabilistic object representation can provide a more robust basis for risk analysis. Furthermore, dynamic risk assessment is only useful if environmental context is available during operation, which makes real-time or near-real-time scene graph generation a central requirement. Therefore, the relevance of this work lies in improving the practical applicability of risk-annotated scene graph generation for autonomous systems by addressing portability, scalability, uncertainty representation, and near-real-time operation.

1.3. Goal of the Thesis

The goal of this thesis is to extend an existing risk-annotated scene graph generation framework so that it can better support dynamic risk assessment in robotic systems. The work focuses on improving the framework in terms of portability, scalability, uncertainty handling, and near-real-time applicability.

First, the existing FastAPI-based implementation is ported to ROS 2 to improve integration with robotic software architectures. This allows the framework to communicate more directly with perception, localization, mapping, and planning components used in autonomous robotic systems. Second, the scene graph generation process is extended using SLAM-based spatial data and loop closure detection in order to support larger environments and maintain spatially consistent information over time.

Furthermore, a probabilistic object representation is developed to aid the risk analysis mechanism. The output of the vision-language model is also refined to produce bounded and consistent scene graph information. Since dynamic risk assessment requires timely environmental context, near-real-time performance is considered throughout the design and implementation of the extended framework.

Finally, the resulting framework is evaluated using multiple datasets, including a dataset collected with a Unitree Go1 robot in a university environment.

2. Theoretical Background

2.1. Autonomous Systems and Safety

Autonomous systems (AS) can be understood as systems that are capable of making decisions independently from direct human control [4]. In robotic systems, this autonomy is typically achieved through a combination of sensing, perception, planning, decision-making, and actuation. For example, an autonomous mobile robot may perceive obstacles, select an alternative path, and continue its task without requiring continuous human input. This ability allows autonomous robots to operate more flexibly in dynamic and partially unknown environments. However, it also makes safety assurance more complex, because the system behaviour is no longer determined only by predefined commands, but also by runtime perception, environmental conditions, and decision-making processes.

Safety is a fundamental requirement in AS because robots interact with humans, physical objects, infrastructure, and the surrounding environment. In dependability theory, safety is commonly defined as the absence of catastrophic consequences for users and the environment [5]. In the context of autonomous robotic systems, safety therefore refers not only to whether the robot performs its intended task, but also to whether it performs the task without creating unacceptable risk. This distinction is important because a robot may be functionally successful while still being unsafe. For example, a robot arm in a shared workspace may complete a pick-and-place task correctly, but the system cannot be considered safe if its motion exposes a nearby human operator to an unacceptable collision risk.

From a risk assessment perspective, safety is closely related to the identification of hazards, the estimation of risk, and the implementation of suitable risk reduction measures [3]. In robotic applications, this requires considering both the severity of possible harm and the likelihood that a hazardous situation may occur. Robot-specific safety standards further define requirements and protective measures for industrial and collaborative robotic systems [6]–[8]. However, safety in AS is strongly dependent on operational context. A motion, speed, or decision that is safe in one situation may become hazardous when a human enters the workspace, when an object is misplaced, or when perception uncertainty increases. Therefore, safety in autonomous robotic systems must be understood as a property that depends on both the technical system and the changing environment in which it operates.

2.1.1. Human-Robot Collaboration

Industry 5.0 is shifting industrial development toward a more human-centric, sustainable, and resilient model, while continuing to support productivity and worker well-being [9]. Within this perspective, Human-Robot Collaboration (HRC) becomes an important requirement for using robotic technologies to support and empower workers rather than simply replacing human labour [9]. Collaborative robotic systems can assist humans in repetitive, strenuous, or hazardous tasks, thereby improving workplace flexibility, ergonomics, and the distribution of work between humans and machines. In industrial settings, HRC has also been discussed as a response to skill gaps, skilled-worker retention, and the need to make manufacturing work more attractive to younger generations [10]. This makes HRC a relevant foundation for future autonomous systems, where robots increasingly need to interpret human presence, environmental changes, and task-specific context during operation.

However, HRC also introduces important challenges in industrial settings. Industrial HRC must address three closely related aspects: ensuring human safety, maintaining trust in automation, and preserving productivity [10]. Among these, human safety remains a primary concern because close physical interaction increases the possibility of unintended human-robot collisions, unexpected robot motion, or interruptions in the robot's behaviour. Such situations can increase the risk of injury and may negatively affect the human operator's trust in the system. At the same time, safety measures must be balanced with productivity requirements, since overly restrictive measures such as excessive speed reductions, large separation distances, or frequent safety stops can reduce system efficiency and interrupt the intended collaborative workflow [10]. The relationship between safety, trust in automation, and productivity is illustrated in Figure 2.1. This trade-off shows that HRC cannot be addressed only as a productivity or automation problem; it also requires systematic risk assessment methods that can identify hazardous situations and define suitable risk reduction measures.

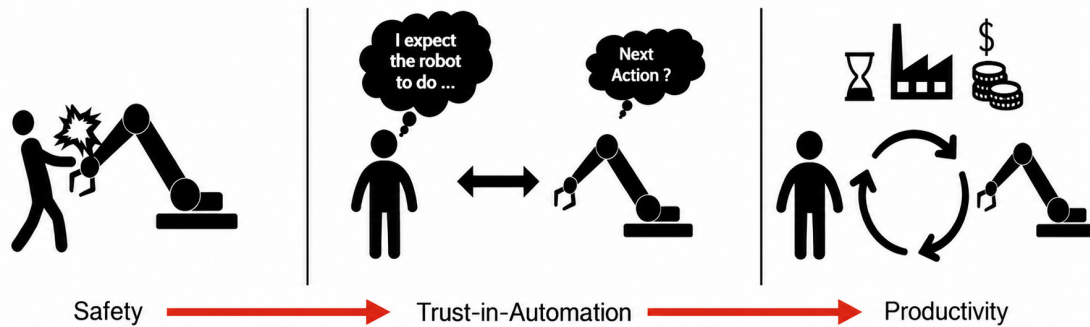


Figure 2.1.: Main challenges in HRC[10].

2. Theoretical Background

2.1.2. Conventional Risk Assessment and Risk Reduction

Conventional risk assessment in robotic systems is generally performed before deployment and is based on the intended use, foreseeable misuse, system limits, operating environment, and known hazards of the system. According to ISO 12100, the risk assessment process includes hazard identification, risk estimation, and risk evaluation, followed by the selection of suitable risk reduction measures [3]. In the context of industrial and collaborative robots, this process is supported by robot-specific safety standards that define requirements for robot systems, collaborative applications, and protective measures [6]–[8].

Risk reduction is usually implemented through a combination of inherently safe design, technical protective measures, and information for use. In HRC, this can include limiting robot speed and force, defining safe separation distances, using protective stops, and applying physical or electronic safeguards around the robot workspace. These measures are essential because they provide a systematic way to reduce the likelihood or severity of hazardous human-robot interactions before the system is put into operation.

However, conventional risk assessment is mainly based on assumptions made during design and deployment. These assumptions include expected human behaviour, known object configurations, defined workspaces, and predefined task conditions. As a result, conventional risk assessment is effective for identifying known and reasonably foreseeable hazards, but it becomes limited when the operating context changes significantly during runtime. This limitation is especially relevant for autonomous robotic systems, where the robot may operate in dynamic environments and make decisions based on sensor data and perception results.

2.1.3. Existing Safety Measures in HRC

In industrial HRC, risk reduction strategies are commonly implemented through two main approaches: the use of collaborative robots and the use of physical or electronic safeguarding measures [10]. Collaborative robots, also referred to as cobots, are designed to operate in direct cooperation with humans within a defined shared workspace. Their safety is supported through hardware and software features such as reduced momentum, force sensing, back-drivable actuators, and protective stop functions that interrupt robot motion when unexpected contact or resistance is detected.

The safeguarding-based approach focuses on monitoring the robot workspace so that robot motion can be adapted or stopped before hazardous human-robot collisions occur. In practice, this is achieved using physical and electronic safeguarding devices that define protected zones around the robot work cell. Examples include safeguarded perimeter door switches, light curtains, pressure-sensitive floors, 2-D scanning LiDAR, and 3-D safety vision systems. These mechanisms are important for reducing the risk of human injury, but they often rely on predefined safety zones, fixed distance thresholds, or conservative stopping rules [10].

2. Theoretical Background

A common method used for workspace monitoring is Speed and Separation Monitoring (SSM). In SSM, the distance between the human and the robot is monitored continuously, and the robot speed is reduced or stopped when the separation distance becomes too small. In a static SSM approach, the monitored zones are usually predefined before operation. For example, a robot may operate at full speed when no human is detected in the outer zone, slow down when a human enters a warning zone, and stop when the human enters a protective stop zone[10]. This approach is simple and practical, but it can be conservative because the safety zones are not always adapted to the actual robot motion, human motion, task state, or sensor uncertainty.



Figure 2.2.: Illustrative representation of safety zones in a collaborative robot workspace.

Dynamic SSM addresses this limitation by adapting the protective separation distance or the robot speed according to the current operating situation. Instead of relying only on fixed zones, dynamic SSM considers factors such as the actual distance between the human and the robot, robot velocity, human motion, stopping time, and uncertainty in the sensed position of obstacles or operators. Marvel discusses SSM performance in terms of both safety and productivity, representing collision potential through separation metrics and sensor uncertainty [11]. This shows that SSM is not only a binary stop-or-run mechanism, but also a method whose effectiveness depends on how accurately the system evaluates separation, uncertainty, and the productivity cost of safety actions.

Although dynamic SSM provides a more flexible basis for HRC safety than purely static safety zones, it also requires reliable perception, continuous monitoring, and suitable runtime decision-making. Therefore, existing safety measures show the importance of moving from fixed protective

2. Theoretical Background

assumptions toward context-aware risk assessment mechanisms that can consider the actual operational situation of the robot, humans, and surrounding environment.

2.1.4. Impact of Artificial Intelligence

The previous subsections show that conventional risk assessment and many existing HRC safety measures are based on pre-deployment assumptions, such as intended use, expected human behaviour, known object configurations, predefined workspaces, fixed safety zones, and foreseeable hazardous scenarios. These assumptions are necessary for systematic risk assessment, but they become less reliable when autonomous robots operate in changing environments. Human positions, object locations, obstacles, task conditions, and spatial relationships may vary during operation, meaning that a risk analysis performed before deployment may no longer fully represent the actual operating situation.

The integration of artificial intelligence into robotic perception, planning, or control further increases this limitation. AI-based components can enable object detection, scene interpretation, motion prediction, task planning, and adaptive decision-making. While these capabilities improve flexibility and autonomy, they also make robot behaviour harder to predict completely during design-time safety analysis. Safety assurance of AI-based systems is complicated by dataset dependence, black-box behaviour, explainability, and lifecycle assurance [12]. In robotic systems, this means that perception errors, low-confidence predictions, distribution shifts, previously unseen objects, changing human behaviour, or software updates after deployment can affect system decisions.

As a result, conventional risk assessment should be understood as a necessary baseline rather than a complete solution for adaptive autonomous robotic systems. If the deployed system operates under conditions that were not fully represented during the original assessment, the selected safety measures may become insufficient. Therefore, dynamic and context-aware risk assessment is required to incorporate runtime information such as robot state, human presence, object locations, spatial relationships, perception uncertainty, and environmental changes. Automated and continuous risk assessment has been proposed for software-defined robotic systems, where changes in software or system behaviour may require risk assessment to be updated during operation [13].

At the same time, AI can also support dynamic risk assessment when its outputs are structured, bounded, and interpretable. AI-based perception and reasoning methods can help identify relevant objects, detect environmental changes, classify scene elements, and localise risk-relevant information around the robot. In this thesis, this motivates the use of a spatial risk-annotated scene graph, which can represent objects, spatial relations, uncertainty, and risk-related context in a machine-readable form to support dynamic risk assessment in autonomous robotic systems.

2.2. Robot Operating System

The Robot Operating System (ROS) is an open-source software framework for developing and integrating robotic systems. Despite its name, ROS is not a traditional operating system; it functions as a middleware layer that provides communication mechanisms, software libraries, development tools, and conventions for building modular robot software [14]. In this thesis, ROS is relevant because the risk-annotated scene graph framework must be integrated with continuous RGB-D sensor streams, pose information, SLAM modules, system status information, and downstream risk assessment components. Therefore, ROS provides the conceptual and technical basis for replacing the service-oriented baseline interface with a robot-native integration approach.

2.2.1. ROS as Robotic Middleware

A ROS-based robotic system is typically organised as a collection of nodes. A node is an executable software component that performs a specific function, such as publishing camera data, processing sensor input, estimating robot pose, performing SLAM, planning motion, controlling actuators, or visualising system state. This node-based structure allows complex robotic systems to be decomposed into smaller modules that can be developed, tested, and replaced independently. For example, an RGB-D camera node can publish sensor data, a SLAM node can estimate the robot pose and map, and a scene graph node can use this information to update a structured representation of the environment.

Communication between nodes is performed using messages. A message defines the structure of the exchanged data, such as an image, pose, velocity command, point cloud, or status value. ROS provides standard message types for common robotic data, while also allowing custom message definitions for application-specific information. This is important for robotic integration because different modules can exchange data in a predictable and structured way.

2.2.2. ROS Communication Mechanisms

ROS provides several communication mechanisms for different interaction patterns. Topics are used for continuous data streams and follow a publish-subscribe model. A node publishes messages to a topic, and one or more other nodes subscribe to that topic. This is suitable for sensor data such as RGB images, depth images, LiDAR scans, odometry, IMU data, and system status. In the context of this thesis, RGB-D observations, pose information, and scene graph updates can naturally be represented as topic-based communication because they are produced repeatedly during robot operation.

Services provide request-response communication. A service is useful when one node needs to request a specific operation from another node and receive a result, such as resetting a scene graph, saving the current graph, or requesting the current system state. Unlike topics, services

2. Theoretical Background

are not intended for continuous streaming, but for discrete interactions where a response is expected.

Actions are used for long-running tasks that may require feedback or cancellation. An action consists of a goal, feedback during execution, and a final result. This mechanism is commonly used for navigation tasks, where a robot receives a goal pose, reports progress, and eventually returns whether the goal was reached. In a scene graph framework, actions could be useful for longer operations such as processing a selected dataset sequence or generating a scene graph for a defined exploration segment.

ROS also uses parameters to configure node behaviour. Parameters can define topic names, thresholds, model settings, update rates, frame identifiers, or other runtime options. This allows the same node to be reused in different experimental setups without changing the source code. Launch files are then used to start multiple nodes together with their required parameters, making it easier to reproduce complete robotic system configurations.

2.2.3. ROS 1 and ROS 2

ROS 1 was the first widely adopted version of ROS and became an important standard for robotics research and prototyping. In ROS 1, node discovery is coordinated through a central ROS Master, which allows publishers and subscribers to find each other before communicating. This architecture supported rapid development and experimentation, but it was mainly designed for research-oriented systems and did not fully address several requirements of modern robotic deployment, such as reliable distributed execution, multi-robot communication, real-time behaviour, security, and production-oriented system design.

ROS 2 was developed to address these limitations and to support more robust robotic applications [15]. A major architectural difference is that ROS 2 uses the Data Distribution Service (DDS) as its communication middleware. DDS enables decentralised discovery, configurable Quality of Service (QoS), and improved support for distributed robotic systems. This means that ROS 2 nodes can discover each other without relying on a central master, which improves suitability for multi-robot systems and distributed deployment.

Quality of Service is an important feature of ROS 2 communication. QoS settings allow developers to control how messages are delivered, for example whether communication should prioritize reliability or low latency. This is important because different robotic data streams have different requirements. A camera stream may prefer recent data and tolerate dropped frames, while system status or safety-related information may require more reliable delivery. ROS 2 also improves support for lifecycle management, embedded platforms, real-time control, and security compared with ROS 1 [15]. At the same time, practical studies of ROS 2 robotic control show that latency and jitter still need to be evaluated carefully in distributed and time-sensitive applications [16].

In the context of this thesis, ROS 2 is relevant because the risk-annotated scene graph framework must operate as part of a robotic system rather than as an isolated external service. RGB-D data,

2. Theoretical Background

pose information, system status, and scene graph updates need to be exchanged continuously between different software components. ROS 2 provides a robot-native communication structure for this purpose, allowing the framework to be integrated with perception sensors, SLAM modules, robot state estimation, visualization tools, and downstream risk assessment mechanisms.

2.3. Spatial Scene Graphs

2.3.1. Scene Graphs

A scene graph is a structured representation of a visual scene in which objects are represented as nodes and relationships between objects are represented as edges [17]. Instead of describing an image or environment only as a collection of detected objects, a scene graph also captures how these objects are related to each other. For example, a scene may contain nodes such as *person*, *chair*, and *table*, while edges may describe relations such as *sitting on*, *next to*, or *under*. Johnson et al. demonstrated the usefulness of scene graphs for image retrieval by representing image content through objects, attributes, and relationships rather than relying only on low-level image features or isolated object labels [17]. The main advantage of a scene graph is that it provides a

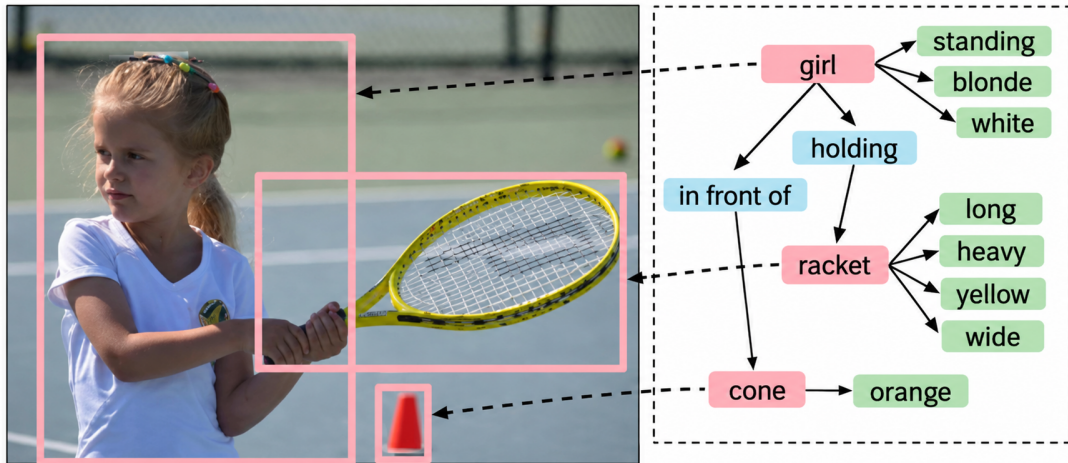


Figure 2.3.: Scene graph example[17].

machine-readable representation of both entities and context. Object detections alone can indicate which objects are present, but they do not fully describe the structure of the scene. In contrast, a scene graph can represent object attributes, spatial arrangements, and semantic relationships in a compact graph format [17]. This makes it useful for tasks that require reasoning about the environment, because downstream modules can query not only whether an object exists, but also how it is positioned or related to other objects.

2. Theoretical Background

Scene graphs are also advantageous because they provide an interpretable bridge between perception and higher-level reasoning. Raw sensor data such as images or point clouds is difficult for symbolic reasoning modules to use directly. A scene graph abstracts this raw data into nodes, edges, and attributes that can be inspected, updated, and processed algorithmically. This is especially useful in robotic systems, where perception results must often be connected to planning, navigation, manipulation, or risk assessment.

In the context of this thesis, scene graphs are relevant because dynamic risk assessment requires more than object detection. Risk may depend on object identity, spatial proximity, support relations, containment, occlusion, human presence, and the current configuration of the environment. A graph-based representation allows such information to be stored in a structured form. When extended with spatial information, uncertainty estimates, and risk annotations, the scene graph becomes a suitable basis for representing the current operational context of an autonomous robotic system.

2.4. Hydra-Based Spatial Scene Graph Generation

Hydra is a real-time spatial perception system for constructing and optimizing 3D scene graphs from robotic sensor data [18]. It is relevant to this thesis because it addresses several requirements that are also important for spatial risk-annotated scene graph generation, including incremental construction, scalability to larger environments, loop closure correction, and near-real-time operation. While the baseline risk-annotated scene graph framework focuses on semantic and risk-related interpretation of RGB-D observations, Hydra provides a spatial backbone for building persistent environment-level representations.

2.4.1. Input Data and Processing Overview

Hydra is designed to operate on continuous sensor data from a moving robot. The main inputs are visual data and inertial measurements. In the Hydra architecture, image data and IMU data are processed by low-level perception modules to support visual-inertial odometry, semantic segmentation, and depth reconstruction [18]. The odometry estimate provides the robot trajectory, while depth and semantic information are used to construct metric-semantic spatial representations of the environment.

The processing pipeline can be understood as a layered transformation from raw sensor data to a structured spatial graph. First, low-level perception estimates robot motion and produces depth and semantic information. Second, mid-level perception constructs local geometric and semantic representations, including a metric-semantic mesh, object nodes, and a places graph. Third, high-level perception performs loop closure detection, scene graph optimization, and room detection. The final output is a persistent layered 3D scene graph that represents the environment at multiple levels of abstraction.

2. Theoretical Background

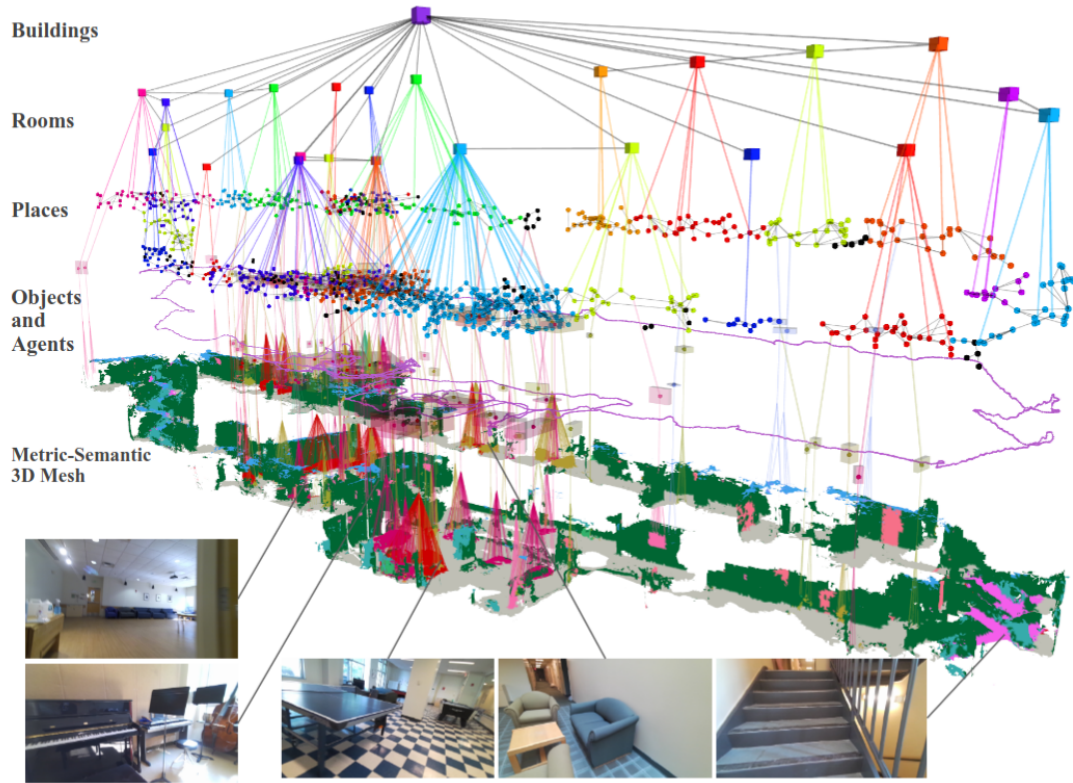


Figure 2.4.: Example of a layered 3D scene graph generated by Hydra, showing abstraction levels from metric-semantic 3D mesh to objects and agents, places, rooms, and buildings. Reproduced from [18].

2. Theoretical Background

Hydra does not use sensor data only to describe the current camera view. Instead, observations are integrated over time to build a persistent representation of the explored environment. This is important because a mobile robot must reason about spaces, objects, and places that may no longer be visible in the current frame. In this way, Hydra connects continuous perception with long-term spatial representation.

2.4.2. Layered 3D Scene Graph Representation

The main output of Hydra is a layered 3D scene graph. In this representation, nodes represent spatial concepts at different levels of abstraction, and edges represent relationships between those concepts. Hydra uses a hierarchy consisting of a metric-semantic 3D mesh, objects and agents, places, rooms, and buildings [18]. This structure allows the environment to be represented not only as geometry, but also as a semantic and topological model.

The lowest layer is the metric-semantic 3D mesh. This layer represents the physical geometry of the environment while also associating semantic labels with the reconstructed surfaces. Above this, the object and agent layer represents physical objects and the robot trajectory. Object nodes can include semantic labels, centroids, and bounding boxes, while the agent is represented by a pose graph describing the robot’s motion through the environment.

The places layer is a sparse topological representation of free space. A place represents an obstacle-free location in the environment, and an edge between two places indicates that straight-line traversal between them is possible. The room layer groups places into larger spatial regions, and the building layer connects the room-level representation into a single environment-level abstraction. This hierarchy is useful because different robotic tasks require different levels of detail. Local navigation can use metric geometry, topological planning can use places, semantic reasoning can use objects and rooms, and high-level decision-making can use the full layered graph.

2.4.3. Metric-Semantic Mesh and Object Layer Construction

Hydra constructs the metric-semantic 3D mesh incrementally rather than building a complete global map in a batch process. A key design choice is the use of a local active window around the robot. Within this active window, Hydra maintains volumetric spatial representations such as a Truncated Signed Distance Field (TSDF) and an Euclidean Signed Distance Function (ESDF). The use of a local window reduces memory usage because the system does not need to maintain an ESDF of the entire environment at all times [18].

The metric-semantic mesh is extracted from the local volumetric representation. Semantic information is integrated with the geometry so that parts of the mesh can be associated with semantic classes. Object nodes are then extracted from the metric-semantic mesh by clustering mesh vertices that belong to the same semantic class. For each putative object, Hydra estimates object-level properties such as a centroid and bounding box. If a newly observed object overlaps

2. Theoretical Background

with an existing object node of the same semantic class, it can be merged with the existing object node. Otherwise, it is inserted as a new object node.

This process is important for spatial scene graph construction because it moves from dense geometric data to compact object-level entities. Instead of only storing surfaces or point clouds, the graph can represent objects as discrete semantic nodes with spatial properties. For this thesis, such object-level information can later be extended with risk annotations, uncertainty values, or VLM-derived semantic properties.

2.4.4. Places and Topological Free-Space Representation

The places layer is one of the most important components of Hydra for robotic spatial reasoning. A place can be understood as an obstacle-free location in the environment that belongs to the navigable free space. Edges between place nodes represent traversability, meaning that the robot can move between connected places without crossing an obstacle. Therefore, the places layer functions as a sparse topological map of the environment.

Hydra extracts places from the ESDF using a Generalized Voronoi Diagram (GVD). The ESDF stores the distance from each voxel to the nearest obstacle. The GVD consists of voxels that are approximately equidistant from multiple obstacles. Intuitively, this creates a skeleton of the free space, because points on the GVD lie near the centre of corridors, rooms, and passages rather than close to walls or obstacles [18]. This makes the GVD useful for extracting topological structure from dense geometric information.

After the GVD is computed inside the active window, Hydra sparsifies it into a graph of places. A subset of GVD voxels is selected as place nodes, and edges are added to represent connectivity between them. Candidate nodes are selected based on geometric and topological conditions, such as the number of obstacle basis points or whether the local neighbourhood matches a corner-like structure. Edge construction then connects nearby place nodes through the GVD structure. If a straight-line edge deviates too much from the GVD path, an intermediate node is inserted to better preserve the shape of the free-space skeleton.

This places graph is useful because it is much smaller than a dense map while still preserving important navigational structure. For spatial risk assessment, the places layer can provide context about where the robot, objects, and humans are located relative to traversable regions, corridors, rooms, and passages. It can also support reasoning about whether a risky object is located in a navigable area, near a doorway, or in a region that affects robot motion.

2.4.5. Room Detection Algorithm

Hydra constructs the room layer from the places graph. The main idea is that rooms can be detected by analysing how free-space connectivity changes when obstacles are virtually inflated. When obstacles are dilated, narrow openings such as doors gradually close. As these openings

2. Theoretical Background

close, the free-space graph separates into connected components, and these components can correspond to individual rooms [18].

The algorithm operates on the places graph rather than directly on the full dense map. Each place node stores a distance to its closest obstacle. Therefore, virtual obstacle dilation can be implemented by removing place nodes whose obstacle distance is smaller than a chosen dilation distance. If the dilation distance is denoted by δ , then all places with obstacle distance below δ are removed from the graph because they would no longer remain in free space after dilation.

Hydra applies several dilation distances and analyses the resulting pruned places graphs. For each dilation distance, the algorithm counts the connected components in the pruned graph. A suitable dilation should cause rooms to become separated while preserving the main structure of each room. To reduce sensitivity to one specific parameter choice, Hydra computes the median number of connected components across multiple dilation distances and selects a graph whose number of components matches this value. These components form the initial room candidates.

However, some place nodes may be removed during dilation and therefore remain unlabeled. Hydra assigns these remaining nodes using a partially seeded graph clustering method based on modularity. The connected components obtained after dilation are used as initial room seeds, and unlabeled nodes are assigned to the room community that best improves graph modularity. This allows the system to obtain room-level segmentation from the sparse places graph without requiring a full dense room segmentation process.

This room detection method has two important advantages. First, it reasons over 3D free space rather than relying only on a 2D floor-plan projection, which can reduce the influence of clutter. Second, because it operates on the sparse places graph, it is more scalable than processing the entire dense map. However, the method is mainly topological. It can fail in open-plan environments where different semantic rooms are not separated by clear geometric boundaries. This limitation is important for this thesis because risk-relevant areas may sometimes need semantic labels, such as laboratory, corridor, workspace, or storage area, which are not directly provided by the room detection algorithm alone.

2.4.6. Loop Closure Detection and Scene Graph Optimization

A major challenge in mobile robot mapping is pose drift. As the robot moves through an environment, small odometry errors accumulate and can cause the estimated map, object locations, and graph structure to become spatially inconsistent. Loop closure occurs when the robot recognises that it has returned to a previously visited area. Hydra uses loop closure not only to correct the robot trajectory, but also to correct the complete 3D scene graph [18].

Hydra performs loop closure detection using hierarchical descriptors derived from multiple graph layers. For each agent node in the robot trajectory, the system stores a keyframe and computes descriptors that describe the surrounding scene. These descriptors include low-level visual appearance, nearby object semantics, and geometric information from the places layer. Loop

2. Theoretical Background

closure detection proceeds in a top-down manner: the system compares higher-level descriptors and then moves through object and visual descriptors to identify possible matches.

After a possible loop closure is found, Hydra performs bottom-up geometric verification. If visual feature matching is sufficient, the relative pose between the current and previous observations can be estimated from image features. If visual verification fails, object-based registration can be attempted using object-level correspondences. This is useful because visual appearance alone can be unreliable under viewpoint changes, illumination changes, or texture-poor environments.

Once a loop closure is accepted, Hydra optimizes the scene graph using an embedded deformation graph. The deformation graph includes selected parts of the full scene graph, such as the agent pose graph, mesh control points, and a sparse subset of the places layer. The optimization corrects spatial drift and then propagates the correction to the remaining graph layers. Object centroids and bounding boxes are recomputed after deformation, overlapping nodes are merged, and rooms can be re-detected from the corrected places graph. This is important because a layered scene graph must remain consistent across all layers. Correcting only the robot trajectory would not be sufficient if objects, places, rooms, and mesh geometry remained misaligned.

2.4.7. Real-Time Architecture and Graph Visualization

Hydra uses a highly parallelized architecture that separates perception into low-level, mid-level, and high-level processes [18]. Low-level perception operates close to sensor rate and includes tasks such as feature tracking, visual-inertial odometry frontend processing, semantic segmentation, and depth reconstruction. These processes operate on the latest sensor data and must run quickly to support continuous robot operation.

Mid-level perception runs at a lower rate and constructs local spatial structures. This includes the visual-inertial odometry backend, metric-semantic mesh construction, places subgraph construction, object detection, and the scene graph frontend. These modules integrate information over a short time horizon and build an odometric version of the scene graph. High-level perception runs more slowly because it depends on larger parts of the accumulated map. This includes loop closure detection, scene graph backend optimization, and room detection.

This separation is important because global optimization and room detection may become more expensive as the map grows. By separating fast local processing from slower global correction, Hydra prevents expensive operations from blocking the sensor-rate perception pipeline. This “thinking fast and slow” design is highly relevant to this thesis because risk-annotated scene graph generation must also balance semantic richness, VLM processing, spatial consistency, and near-real-time operation.

Hydra visualizes the resulting 3D scene graph as a layered spatial representation. The metric-semantic mesh forms the base of the representation, while objects, agents, places, rooms, and buildings are visualized as graph layers above it. Edges show relationships within and across layers, such as object-to-place, place-to-room, and room-to-building connections. This visualization is useful for qualitative inspection because it allows the user to check whether the mesh aligns

2. Theoretical Background

with the environment, whether objects are placed correctly, whether the places graph captures navigable structure, and whether rooms are segmented meaningfully.

2.4.8. Adaptation for RGB-D and IMU Input from a Real Robot

If Hydra is used with RGB-D and IMU data from an actual robot, several practical adaptations are required. First, the sensor streams must be synchronized. The colour image, depth image, IMU measurements, and pose estimate must refer to consistent time intervals. If the data are not time-aligned, the reconstructed 3D structure may become distorted because depth measurements and pose estimates may correspond to different robot positions.

Second, camera calibration and frame transformations must be available. The RGB-D camera intrinsics are required to project depth pixels into 3D points. The extrinsic transformation between the RGB camera, depth camera, IMU, and robot base frame must also be known. In a ROS 2-based system, these transformations are usually represented through the `tf` tree. Without correct transformations, object positions and spatial graph nodes may be placed incorrectly in the global map.

Third, the depth image must be valid and aligned with the colour image or semantic segmentation output. If semantic labels are predicted from the RGB image, but 3D points are generated from an unaligned depth image, semantic labels may be assigned to the wrong geometry. Therefore, either the depth image should be registered to the RGB frame, or the system must explicitly transform between the RGB and depth camera frames.

Fourth, Hydra requires a source of odometry or visual-inertial odometry. In the original system, visual-inertial odometry and external odometry sources are used to estimate the robot trajectory. For a real robot, this could come from an RGB-D visual-inertial odometry pipeline, wheel odometry fused with IMU, an external SLAM system, or another state-estimation module. The quality of this pose estimate directly affects the quality of the spatial scene graph. Loop closure can correct accumulated drift, but poor odometry may still degrade local mapping and object association.

Fifth, a semantic segmentation source is required if the original Hydra-style metric-semantic mesh construction is used. Hydra assumes semantically labelled visual input for assigning semantic labels to reconstructed geometry. In the proposed thesis framework, this part may need to be adapted because semantic object interpretation is provided by SAM, RAP, and VLM-based reasoning rather than by a conventional pixel-wise semantic segmentation network. Therefore, one design option is to use Hydra for geometric and topological spatial structure, while the existing risk-annotated scene graph pipeline provides object-level semantic and risk information.

Finally, the output representation must be adapted so that Hydra’s spatial graph can communicate with the risk-annotated scene graph. This requires mapping Hydra entities such as objects, places, rooms, and robot poses into the graph representation used by the risk assessment pipeline. For example, Hydra can provide global object position, room association, and loop-closure-corrected spatial consistency, while the VLM-based framework can provide object descriptions, semantic

2. Theoretical Background

relations, risk metadata, and uncertainty estimates. The practical integration therefore requires careful definition of shared node identifiers, coordinate frames, timestamps, and update rules.

2.4.9. Relevance to the Proposed Framework

Hydra provides a strong methodological reference for this thesis because it demonstrates how a robot can construct a persistent spatial scene graph in real time. The most relevant ideas are the layered graph representation, incremental construction of spatial layers, local active-window mapping, places extraction from free-space topology, room detection from the places graph, loop closure correction, and separation between fast local processing and slower global optimization.

However, Hydra and the proposed thesis framework have different primary goals. Hydra focuses on real-time spatial perception and the construction of a globally consistent 3D scene graph. The proposed thesis framework focuses on extending an existing risk-annotated scene graph pipeline so that it can support robotic integration, larger environments, uncertainty representation, VLM-based semantic interpretation, and dynamic risk assessment. Therefore, Hydra can be used as a spatial backbone or design reference, while the risk-annotated scene graph framework contributes the semantic and risk-oriented interpretation layer.

In this thesis, the intention is to reuse the spatial mapping and scene graph construction principles from Hydra while extending the representation with risk-related information. The resulting framework should preserve the advantages of Hydra, such as spatial consistency, scalability, loop closure correction, and real-time architectural separation, while adding the ability to represent risk-relevant objects, uncertainty, and semantic relations required for downstream dynamic risk assessment.

2.5. Risk-Annotated Scene Graph Framework Baseline

This section introduces the existing risk-annotated scene graph framework that forms the basis of this thesis [19]. The purpose is to explain the conceptual pipeline of the original work, including how visual input is processed, how objects and relationships are extracted, and how risk-related information is represented in the generated scene graph. This provides the necessary background for understanding the extensions proposed in this thesis.

The discussion focuses on the high-level concepts of the original framework rather than implementation-specific details. In particular, it describes the role of perception, segmentation, retrieval-augmented perception, vision-language model reasoning, scene graph construction, and risk annotation. The limitations of the existing framework are also considered, especially with respect to portability, scalability, spatial consistency, uncertainty representation, and runtime applicability.

2.5.1. Information Flow in the Baseline Framework

The baseline framework follows a modular processing pipeline in which RGB-D data is received, processed, interpreted, and integrated into a risk-annotated scene graph. The input to the system is an RGB-D observation of the environment, consisting of a colour image and its corresponding depth image. In the original implementation, this input can be provided through a dataset or submitted through an external interface. The external communication is handled through a FastAPI-based interface, which allows incoming samples and processing requests to be passed into the internal pipeline. This design separates data input and client communication from the perception and scene graph generation modules.

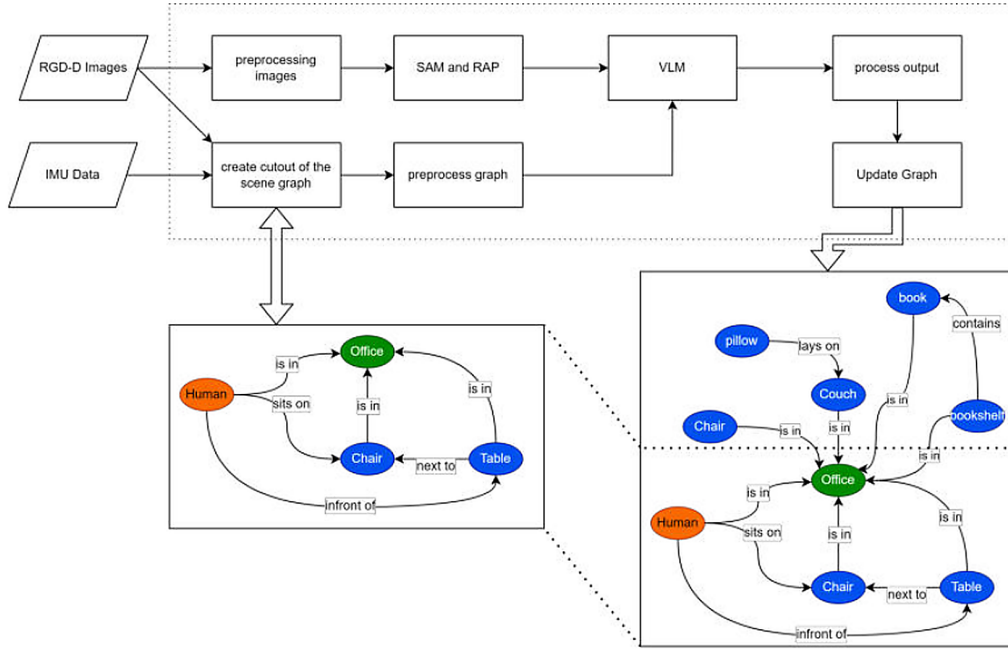


Figure 2.5.: Information flow of the existing risk-annotated scene graph framework. Adapted from [19].

After an RGB-D input sample is received, the frame is handled as a task in the processing pipeline. The colour image is first used for object segmentation. For this purpose, the framework uses the Segment Anything Model (SAM) to generate object masks or image regions that may correspond to relevant entities in the scene. These masks are then used to create image cutouts for detected object candidates. The associated depth image, camera parameters, and pose information support later spatial processing, including coordinate conversion, visibility checking, and the spatial grounding of graph elements.

The segmented object candidates are then processed using a Retrieval-Augmented Perception (RAP) mechanism and a Vision-Language Model (VLM). RAP retrieves relevant or similar labels from a visual knowledge base for the detected object cutouts. This provides additional label candidates and prior visual context for the later interpretation stage. In parallel, the existing

2. Theoretical Background

scene graph is reduced to a cutout that contains only the nodes and edges relevant to the current camera view. This scene-graph cutout is generated using the current pose and depth image, so that only likely visible graph elements are passed forward for inference.

The VLM receives the RGB image together with the scene-graph cutout and the possible labels retrieved for the detected object candidates. It is then used to interpret the current scene and produce structured information about visible entities, semantic labels, relationships, confidence values, and risk-relevant properties. The VLM output is parsed into a graph-like representation and then merged with the existing scene graph. During this update, existing nodes and edges can be modified, newly detected entities can be added, and graph information can be updated according to the latest observation.

Finally, the extracted and updated information is maintained as a risk-annotated scene graph. In this representation, relevant objects are modelled as nodes, while spatial, semantic, or risk-related relationships are represented as edges or attributes. The depth component of the RGB-D input is important because it supports spatial grounding of detected objects and enables approximate geometric relationships to be represented in the graph. The resulting graph provides a machine-readable representation of the observed scene and can be used as structured input for downstream reasoning or risk assessment. The overall information flow can therefore be summarised as RGB-D input, task handling, object segmentation, object cutout generation, RAP-based label retrieval, scene-graph cutout generation, VLM-based interpretation, and scene-graph update.

The main design elements of the baseline framework are its modular worker-based structure, its separation between external communication and internal processing, its use of foundation models for segmentation and semantic interpretation, its retrieval-based support for object labelling, and its graph-based representation of RGB-D scene information. These elements make the framework suitable as a basis for risk-aware scene understanding.

2.5.2. FastAPI

FastAPI is a modern Python web framework used for building application programming interfaces (APIs) [20]. It allows external clients to communicate with a software system through network-accessible endpoints, typically using HTTP requests. In robotics and perception systems, this can be useful when a perception pipeline is executed as a separate service and must receive RGB-D observations, processing requests, or configuration information from external programs.

One advantage of FastAPI is that structured interfaces can be developed quickly in Python. The expected format of incoming requests can be defined explicitly, which is useful when external clients submit complex data such as RGB-D images, timestamps, and pose information. FastAPI also supports asynchronous web-server operation, making it suitable for service-based applications that handle multiple requests, communicate with external processes, or expose machine-learning components through a network interface [20]. In the baseline risk-annotated scene graph framework, this provides a practical way to expose the scene graph generation pipeline as an external service while keeping the client application separated from the internal perception, segmentation, VLM, and scene graph modules.

2. Theoretical Background

However, a FastAPI-based interface also has limitations for direct integration with autonomous robotic systems. Robotic software usually requires continuous sensor streams, low-latency communication, synchronised multi-modal data, distributed robot nodes, and predictable timing behaviour. A REST-based API is well suited for request-response communication [20], but it is less natural for high-frequency RGB-D streams and real-time control loops. Therefore, FastAPI is useful as a modular service interface in the baseline framework, but it is not ideal as the primary communication mechanism for portable robotic deployment. This motivates the later porting of the framework to ROS 2, where RGB-D data, pose information, scene graph updates, and system status can be handled using robot-native communication mechanisms.

2.5.3. SAM-Based Object Segmentation

The Segment Anything Model (SAM) is a foundation model for image segmentation introduced by Kirillov et al. [21]. It is designed as a promptable segmentation model that can generate masks from inputs such as points, bounding boxes, or masks. Since SAM is not limited to a fixed set of object classes, it is useful for open-ended visual perception tasks where the objects in a scene may not be known in advance.

In the baseline risk-annotated scene graph framework, SAM is used to extract candidate object regions from the RGB component of the RGB-D input. For each retained object candidate, the framework keeps an object crop, a bounding box, a binary segmentation mask, and a confidence-related score. The crop is used for later visual interpretation, the bounding box and mask describe the image-space extent of the object candidate, and the score indicates the quality or stability of the segmentation.

SAM therefore provides the low-level object proposal stage of the pipeline. However, it does not provide object identities, semantic relationships, risk labels, physical meaning, or 3D positions. Depth data, retrieval-based labels, and VLM-based interpretation are required before the segmented regions can be represented as meaningful nodes and relations in a risk-annotated scene graph.

2.5.4. Retrieval-Augmented Perception

Retrieval-Augmented Perception (RAP) in the baseline framework follows the broader idea of retrieval-augmented methods, where stored information is retrieved and used to support the interpretation of new input. This idea is closely related to retrieval-augmented generation, introduced by Lewis et al. for knowledge-intensive natural language processing tasks, where generated outputs are supported by information retrieved from an external knowledge source [22]. In the baseline framework, this principle is transferred from text-based generation to visual perception. Instead of interpreting each segmented object only from the current RGB-D observation, the system retrieves relevant information from previously stored visual observations and uses it as additional context for object interpretation.

2. Theoretical Background

After SAM generates candidate object regions, these object cutouts are passed to the RAP mechanism. RAP compares the current visual object candidates with entries in a visual knowledge base and retrieves similar or relevant object labels. These retrieved labels act as prior information for the later VLM-based interpretation stage. In this way, RAP supports object consistency across observations and reduces the dependence on the VLM having to infer every object label from the current image alone.

The main advantage of RAP is that it introduces memory-like behaviour into the perception pipeline. If an object or a visually similar object has been observed before, the retrieved information can help guide the interpretation of the current scene. This is useful for risk-annotated scene graph generation because object identity and consistency are important when maintaining a structured representation of the environment over time. However, RAP does not independently solve object recognition or risk assessment. Its output must still be interpreted together with the current RGB-D observation, the segmented object regions, and the VLM output before the information can be inserted into the scene graph.

2.5.5. Vision-Language Model Reasoning

Vision-Language Models (VLMs) are multimodal models that combine visual and textual information to support image understanding through natural language. Unlike conventional object recognition models that predict labels from a fixed set of classes, VLMs can interpret visual content using prompts, descriptions, or structured instructions. Instruction-following VLMs such as LLaVA demonstrate how visual inputs can be connected with language-model reasoning through multimodal instruction tuning [23]. This makes VLMs useful for open-ended scene understanding, where the system must identify objects, infer relationships, and reason about context beyond predefined categories.

Within the baseline framework, the VLM acts as the semantic reasoning component. It converts visual and contextual information into structured scene-level outputs, including object identities, attributes, relationships, confidence values, and risk-relevant descriptions. These outputs provide the semantic information required for constructing and updating the risk-annotated scene graph.

The baseline study evaluated LLaVA 1.6 Mistral 7B, Qwen 2.5-VL 7B, and Phi-4 Vision Multimodal for the scene graph generation task. The comparison considered input- and output-format compatibility, repetition and hallucination behaviour, and the stability of location-format output. Based on this evaluation, the Qwen model series was identified as the best performing option for the baseline framework [19]. Later testing used Qwen 3-VL to use the newer available model version while preserving the framework’s model-interchangeable design [19].

However, VLM-based reasoning can produce inconsistent descriptions, miss small objects, or infer relationships that are not fully supported by the visual input. Therefore, VLM output cannot be treated as directly reliable safety information. It must be bounded, checked, and converted into a consistent graph representation before it can support downstream risk assessment. This motivates the refinement of VLM outputs in this thesis.

2.5.6. Scene Graph Construction and Risk Annotation

The baseline framework represents the scene graph as a directed graph, where entities in the environment are modelled as nodes and relationships between entities are modelled as edges [19]. The original implementation uses NetworkX, a Python library for network analysis and graph manipulation, which supports incremental graph operations such as adding, updating, and removing nodes and edges [24].

For external communication, the internal graph is converted into a dictionary-based representation. This is necessary because a native NetworkX graph object is not directly suitable for robust API transfer. The dictionary representation preserves the logical structure of the graph by storing node and edge information separately, making it easier to return through the FastAPI interface and process by external clients.

Each node stores semantic, spatial, and risk-related information. Important node attributes include a persistent identifier, object label, semantic description, semantic pose, 3D position, risk metadata, appearance score, confidence score, and layer type. The semantic description describes the visual identity of the entity, while the semantic pose describes its current physical state, such as standing, lying, open, closed, or moving. Edges store relationships between nodes, including the source node, target node, relationship label, and distance between the connected entities.

The graph update process must handle perception errors such as false positives and false negatives. To improve robustness, the baseline framework uses an appearance score-based graph combination strategy. When an object is repeatedly observed with consistent properties, its appearance score is increased. When it is missing from the latest VLM output, its score is decreased. A node is removed only when its appearance score falls below a predefined threshold, which prevents isolated missed detections from immediately deleting objects from the graph.

Overall, the scene graph construction strategy combines semantic information, spatial information, and risk-related metadata in a machine-readable representation. This makes the baseline framework suitable for incremental risk-aware scene understanding, while also motivating later extensions for spatial consistency, uncertainty handling, and scalability in larger environments.

2.5.7. Limitations of the Baseline Framework

Although the baseline framework provides a suitable foundation for risk-annotated scene graph generation, it has several limitations when considered for deployment in autonomous robotic systems. The first limitation is related to portability. The original framework is exposed mainly through a FastAPI-based service interface, which is useful for dataset-based testing and external client communication. However, autonomous robotic systems typically rely on robot middleware for continuous sensor streams, synchronised data exchange, distributed nodes, and runtime coordination. As a result, the baseline interface is not directly aligned with robot-native communication patterns, making integration with real robotic platforms less straightforward.

2. Theoretical Background

A second limitation concerns scalability across larger environments. The baseline framework processes RGB-D observations and updates a scene graph based on the current frame, camera pose, and visible graph elements. This is suitable for local scene understanding, but it does not by itself provide a complete mechanism for building and maintaining a spatially consistent graph across multiple rooms, corridors, or revisited locations. As the operating area increases, the system requires stronger spatial anchoring so that objects observed at different positions and times can be associated with the correct location in a global environment representation.

Closely related to this is the absence of a dedicated loop closure mechanism. In mobile robotic operation, the robot may revisit a previously observed area after accumulating pose drift. Without loop closure, the system may treat revisited objects or locations as new observations, or may update the scene graph using spatial information that is no longer globally consistent. This limits the framework’s ability to maintain a reliable long-term scene graph in larger environments. For this reason, integration with SLAM-based spatial data and loop closure detection is necessary for extending the framework beyond local frame-by-frame scene understanding.

The baseline framework also has limitations with respect to uncertainty representation. Object candidates are generated from segmentation, interpreted by a VLM, and then merged into the scene graph using confidence-related and appearance-based information. This supports basic persistence handling, because objects that are repeatedly observed can be treated as more stable than objects that appear only briefly. However, this does not provide a full probabilistic representation of object existence, object class, or spatial position. For downstream risk assessment, such probabilistic information is important because the risk associated with an object may depend not only on its detected class, but also on the likelihood of its presence, the uncertainty of its location, and the reliability of its classification. Representing important objects with probability distributions or uncertainty estimates would allow the risk assessment mechanism to distinguish between confidently detected objects, weakly observed objects, and objects whose position or classification remains uncertain.

Finally, the baseline framework depends strongly on the quality and consistency of VLM output. VLM-generated scene graph information may contain formatting errors, hallucinated relationships, missed objects, or inconsistent descriptions. Although parsing and graph-combination mechanisms can reduce some of these effects, the output still needs to be bounded and checked before it can be used as reliable input for risk assessment. These limitations motivate the extensions proposed in this thesis: porting the framework to ROS 2, integrating SLAM-based spatial information and loop closure, introducing probabilistic object representation, refining VLM output constraints, and considering near-real-time operation during system design.

2. Theoretical Background

2.6. Section4

2.7. Section5

2.8. Section6

2.9. Section7

3. Methodology

3.1. Baseline System and Proposed Extensions

Hydra builds the 3D map and publishes local graph context in real time. The VLM pipeline asynchronously consumes this context together with RGB-D frames, produces object/risk observations, and publishes them back. A Hydra-side fusion module validates and integrates these observations into the global Dynamic Scene Graph.

4. Experiments

5. Conclusions

Bibliography

- [1] International Federation of Robotics, *World robotics 2025: Press conference presentation*, 2025. [Online]. Available: https://ifr.org/downloads/press_docs/PressConference2025_presentation.pdf.
- [2] Stanford Institute for Human-Centered Artificial Intelligence, *Artificial intelligence index report 2025*, 2025. [Online]. Available: <https://hai.stanford.edu/ai-index>.
- [3] International Organization for Standardization, *Iso 12100: Safety of machinery – general principles for design – risk assessment and risk reduction*, International Organization for Standardization, 2010.
- [4] M. Osborne, R. D. Hawkins, and J. A. McDermid, “Analysing the safety of decision-making in autonomous systems,” in *Computer Safety, Reliability, and Security*, ser. Lecture Notes in Computer Science, vol. 13414, Springer, 2022, pp. 3–16. doi: 10.1007/978-3-031-14835-4_1.
- [5] A. Avizienis, J.-C. Laprie, B. Randell, and C. Landwehr, “Basic concepts and taxonomy of dependable and secure computing,” *IEEE Transactions on Dependable and Secure Computing*, vol. 1, no. 1, pp. 11–33, 2004. doi: 10.1109/TDSC.2004.2.
- [6] International Organization for Standardization, *Iso 10218-1:2025 robotics – safety requirements – part 1: Industrial robots*, International Organization for Standardization, 2025. [Online]. Available: <https://www.iso.org/standard/73933.html> (visited on 05/25/2026).
- [7] International Organization for Standardization, *Iso 10218-2:2025 robotics – safety requirements – part 2: Industrial robot applications and robot cells*, International Organization for Standardization, 2025. [Online]. Available: <https://www.iso.org/standard/73934.html> (visited on 05/25/2026).
- [8] International Organization for Standardization, *Iso/ts 15066:2016 robots and robotic devices – collaborative robots*, International Organization for Standardization, 2016. [Online]. Available: <https://www.iso.org/standard/62996.html> (visited on 05/25/2026).

Bibliography

- [9] European Commission, "Industry 5.0: Towards a sustainable, human-centric and resilient european industry," European Commission, 2021. [Online]. Available: <https://op.europa.eu/en/publication-detail/-/publication/468a892a-5097-11eb-b59f-01aa75ed71a1> (visited on 05/25/2026).
- [10] S. P. Kumar, C. Savur, and F. Sahin, "Survey of human–robot collaboration in industrial settings: Awareness, intelligence, and compliance," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 51, no. 1, pp. 280–297, 2021. doi: 10.1109/TSMC.2020.3041231.
- [11] J. A. Marvel, "Performance metrics of speed and separation monitoring in shared workspaces," *IEEE Transactions on Automation Science and Engineering*, vol. 10, no. 2, pp. 405–414, 2013. doi: 10.1109/TASE.2013.2237904.
- [12] A. V. Silva Neto, J. B. Camargo, J. R. Almeida, and P. S. Cugnasca, "Safety assurance of artificial intelligence-based systems: A systematic literature review on the state of the art and guidelines for future work," *IEEE Access*, vol. 10, pp. 130 733–130 770, 2022. doi: 10.1109/ACCESS.2022.3229233.
- [13] P. Grimmeisen, R. Golwalkar, Y. Ma, and A. Morozov, "Automated and continuous risk assessment for ros-based software-defined robotic systems," in *2023 IEEE 19th International Conference on Automation Science and Engineering (CASE)*, IEEE, 2023, pp. 1–7. doi: 10.1109/CASE56687.2023.10260416.
- [14] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "Ros: An open-source robot operating system," in *ICRA Workshop on Open Source Software*, 2009.
- [15] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot operating system 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, eabm6074, 2022. doi: 10.1126/scirobotics.abm6074.
- [16] L. Puck, P. Keller, T. Schnell, C. Plasberg, A. Tanev, G. Heppner, A. Roennau, and R. Dillmann, "Performance evaluation of real-time ros2 robotic control in a time-synchronized distributed network," in *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, IEEE, 2021, pp. 1670–1676. doi: 10.1109/CASE49439.2021.9551447.
- [17] J. Johnson, R. Krishna, M. Stark, L.-J. Li, D. A. Shamma, M. S. Bernstein, and L. Fei-Fei, "Image retrieval using scene graphs," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3668–3678. doi: 10.1109/CVPR.2015.7298990.
- [18] N. Hughes, Y. Chang, and L. Carlone, "Hydra: A real-time spatial perception system for 3d scene graph construction and optimization," in *Robotics: Science and Systems*, New York City, NY, USA, 2022.
- [19] André Schiel, "Online Risk-Annotated Scene Graohs for Autonomous Systems," Master's thesis, University of Stuttgart, Stuttgart, Germany, 2026.

Bibliography

- [20] FastAPI. “Fastapi documentation.” (2026), [Online]. Available: <https://fastapi.tiangolo.com/> (visited on 05/28/2026).
- [21] A. Kirillov, E. Mintun, N. Ravi, *et al.*, “Segment anything,” *arXiv preprint arXiv:2304.02643*, 2023. arXiv: 2304.02643 [cs.CV].
- [22] P. Lewis, E. Perez, A. Piktus, *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [23] H. Liu, C. Li, Q. Wu, and Y. J. Lee, “Visual instruction tuning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023.
- [24] A. A. Hagberg, D. A. Schult, and P. J. Swart, “Exploring network structure, dynamics, and function using networkx,” in *Proceedings of the 7th Python in Science Conference*, Pasadena, CA, USA, 2008, pp. 11–15.

A. Example appendix chapter

Declaration of Compliance

I hereby declare to have written this work independently and to have respected in its preparation the relevant provisions, in particular those corresponding to the copyright protection of external materials. Whenever external materials (such as images, drawings, text passages) are used in this work, I declare that these materials are referenced accordingly (e. g. quote, source) and, whenever necessary, consent from the author to use such materials in my work has been obtained.

Signature:

Stuttgart, on the 30 October 2026